

CS 30700: Sprint Planning 2

BoilerSpace

Team 3: Aagam Bapna, Sheehan Dandapat, Rohan Potta, Nathan Wan, Jed Macapinlac, Bidit Acharyya

Sprint Overview

Distillation: For this sprint, our primary goal is to significantly expand the BoilerSpace platform by delivering a seamless blend of advanced user personalization, real-time social features, and intelligent utilities. We are focused on implementing robust account management and real-time direct messaging, enhancing campus navigation with location-aware routing and room sorting, and integrating AI to generate personalized study guides directly from user notes. Furthermore, we aim to ensure the reliability of live, crowdsourced community data such as room occupancy and snack availability by establishing strict concurrency and spam prevention measures, ultimately providing a dynamic and highly responsive experience for students.

Scrum Master: Sheehan Dandapat

Meeting Plan: Monday/Wednesday/Friday @ 4:30 PM in WALC 3076

Risks and Challenges:

- **Real-Time Data Synchronization:** Implementing seamless WebSocket (Socket.io) integration for direct messaging and ensuring real-time UI updates for room occupancy and crowdsourced snack reports without performance bottlenecks.
- **Cascading Data Deletion:** Safely managing complex account and message deletions. We must ensure that all associated data (messages, notes, check-ins, tokens, and friendships) is completely wiped from MongoDB without leaving orphaned records or crashing active threads.
- **AI Prompt Engineering & Constraints:** Formatting course notes efficiently so the Gemini API accurately generates study guides and practice questions *without* revealing direct answers or hallucinating, all while managing Google Cloud Storage integration.
- **Geolocation & Third-Party APIs:** Gracefully handling edge cases where users deny browser location permissions, while accurately computing Haversine distances and integrating the MapBox Directions API for campus walking routes.
- **Concurrency & Spam Prevention:** Handling concurrent user check-ins and crowdsourced reports (like snack availability) while preventing spam, rapid duplicate submissions, and ensuring the data doesn't become skewed.

Backlog (Cross out what's finished or in progress)

Functional Requirements:

Login/Profile

- ~~1. As a user, I would like to register for a BoilerSpace account so that I can save my preferences and access the full suite of features.~~

- ~~2. As a user, I would like to be able to login and manage my BoilerSpace account so that I can keep my personal data secure and up to date.~~
- ~~3. As a user, I would like to be able to verify my Purdue email by getting an OTP so that the platform remains exclusive to the campus community and prevents spam.~~
- ~~4. As a user, I would like my password to be reset if I forget it so that I can regain access to my account without creating a new one.~~
- ~~5. As a user, I would like to be able to edit my profile (display name, major, year) so that others recognize me in sessions.~~
- ~~6. As a user, I want to block another user so that I cannot receive messages from them and their content is hidden across the platform.~~
7. As a user, I want to set my public/private profile visibility so that I control what others see.
- ~~8. As a user I want to set my classes for this semester, so that others know what I am studying~~
- ~~9. As a user, I want to be able to friend others based on what classes they are taking so that I can build a network of study partners for my specific coursework.~~
- ~~10. As a user, I want to update my availability or preferred study times, so that others know when I'm typically open to studying.~~
11. As a user, I want to manage my notification preferences from my profile, so that I can control how and when I'm contacted.
- ~~12. As a user, I want the option to upload a profile picture so others can recognize me more easily.~~
- ~~13. As a user, I want to be able to delete my account so that I can remove my personal data if I leave the university or stop using the app, and we will also have 2FA.~~

Insights

14. As a user, I want to see my total study time for this week, so that I can track my productivity.
15. As a user, I want to receive achievements for hitting study goals so that I stay motivated to maintain a consistent study routine.
16. As a user, I want AI recommendations based off my study habits so that I can improve efficiency
- ~~17. As a student, I would like to receive notifications for when a room is under a certain capacity so that I can travel to the location knowing I will successfully find a seat.~~
- ~~18. As a student, I want to see a clear explanation when location access is requested, so that I understand why it is needed before granting permission.~~
19. As a student, I want to receive reminders for upcoming study sessions I joined, so that I don't forget to attend.

20. As a student, I want to receive notifications when new content is uploaded to courses, sessions, or events that I follow so that I stay up to date without manually checking the platform.
21. As a student, I want to temporarily mute all notifications, so that I can focus without interruptions.
22. As an admin, I want to deactivate abusive accounts so that the community stays safe, if someone gets reported too many times, we will remove them from our platform.
23. As an admin, I want to verify official campus organizations, so that students can trust event legitimacy.
24. As an admin, I want to remove inappropriate content (notes, comments, or events), so that the platform remains respectful.
25. As a student, I want to confirm or dispute a previous user vibe report so that the live data remains accurate and up to date
- ~~26. As a student, I want to see a timestamp of last update for a room status so I can judge how reliable the information is before I start walking~~
27. As a student, I want to filter the map for quiet zones specifically so I can find a place for deep focus rather than group work.
28. As a student, I want to export an AI generated study guide to PDF or markdown so I can study offline or print it out
29. As a student, I want to see a simple green or red dot next to a room name so I can instantly know if it is likely open or full
- ~~30. As a student, I want to click a single button to check in to a room so I can contribute to the live data without filling out a long form~~
- ~~31. As a student, I want to see a basic list of buildings sorted alphabetically so I can find my favorite study spot quickly~~
- ~~32. As a student, I want to see how many people are currently checked into a specific building so I can gauge the overall crowd level~~
- ~~33. As a student, I want to use a search bar to find a specific building by its name or campus abbreviation~~
- ~~34. As a student, I want my dashboard and room pages to display recent buildings I've visited, key room amenities (such as whiteboards and outlets), and the last updated time of room status so that I can quickly choose a study space that fits my needs and trust the availability information.~~
- ~~35. As a student, I want to filter my course list by the current semester so I can focus only on the classes I am taking right now~~
- ~~36. As a student, I want to see a small icon next to a building name that indicates if it has a cafe or vending machines nearby so I can choose a study spot based on snack availability~~
37. As an admin, I want automatic spam detection so less moderation work is

needed

- 38. As an admin, I want to be able to broadcast announcements so users can receive platform updates
- 39. As an admin, I want to see platform analytics to see which features are most popular, least-used, and make data-driven decisions to improve the platform
- ~~40. As a student looking for a seat, I want to view an interactive campus map so that I can quickly see buildings and rooms.~~
- ~~41. As a student I want directions to different study locations so that I can find the fastest route to a building I haven't visited before.~~
- ~~42. As a student, I want to see a list of rooms sorted by distance from my current location so that I can find the nearest available spot.~~
- ~~43. As a student, I want to bookmark my favorite rooms so that I can quickly check them in the future.~~
- 44. As a student I would like to find people from my class to study with so that we can help each other understand course material.
- 45. As a student, I want to search for certain classes that people are studying for so that I can join a group that is working on the same assignment/project as me.
- 46. As a student, I want to search for certain clubs and if they are hosting any study get-togethers so that I can meet people with shared interests.
- ~~47. As a student, I want to comment on a session's notes so that I can ask clarifying questions or provide corrections to shared content.~~
- 48. As a student, I want to know certain exam dates for certain classes so that I can prioritize my study sessions effectively.
- 49. As a student, I want the option of dark mode so that I can study more comfortably at night.
- 50. As a student, I want to adjust the text size, so that all the content is easier to read.
- ~~51. As a student, I want to view and upload notes from study sessions for my course so that I can both review material I may have missed and contribute to the shared knowledge base.~~
- ~~52. As a student, I want to upvote/downvote certain notes so that the most helpful and accurate resources rise to the top.~~
- 53. As a student, I want to rate and post reviews on study areas so others can find the best spots
- ~~54. As a student, I want to filter rooms based off amenities (whiteboards, outlets) so that I can find a workspace that fits my equipment needs.~~

AI Assistant

- ~~55. As a student, I want to ask AI assistant questions about the aggregated study notes for my course so that I can find specific explanations without reading every~~

document.

56. As a student, I want the AI assistant to show citations that link back to the exact note or page it used, so that answers are traceable and verifiable.

~~57. As a student, I want the AI assistant to generate concise study guides from my notes and highlight key topics, so I can review efficiently and know what to focus on.~~

~~58. As a student, I want the AI to suggest practice questions without giving full solutions, so that I can test my understanding.~~

59. As a student, I want to bookmark helpful AI responses, so that I can easily return to them later.

60. As a student, I want the AI to help create a study plan and schedule so I can organize my time efficiently

61. As a student, I want AI to suggest external resources like textbooks or videos to improve my understanding of certain topics

Clubs / Organizations

~~62. As a club organizer, I want to post events with time and location so that students can discover them easily.~~

63. As a club organizer, I want to tag events by category (callout, workshop, social, fundraiser) so that students can filter them.

64. As a club organizer, I want to create an event with a title, description, date, time, and campus location, so that students clearly understand what the event is.

65. As a club organizer, I want to manage who can create events under my club so that only authorized members can post.

66. As a club organizer, I want to attach external links (sign-up forms, Discord, social media) so that students can engage beyond the event.

67. As a club organizer, I want to see how many students plan to attend so that I can plan resources.

~~68. As a club organizer, I want to post announcements or updates related to events, so that attendees stay informed.~~

69. As a club member I want to RSVP to events so that the organizers know to expect me.

70. As a club organizer, I want to restrict even visibility to certain audiences (member-only or public), so that events are targeted.

~~71. As a club organizer, I want to create and manage a club profile, so that students can learn about the organization.~~

72. As a club organizer, I want to assign different roles to club members (e.g., admin, officer) so that responsibilities are clear.

~~73. As a club organizer, I want to approve or remove club members, so that organizations stay moderated.~~

- 74. As a club organizer, I want to schedule recurring events so members can plan attendance in advance
- 75. As a club member, I want to see files or slides from past events to stay up to date
- 76. As a club organizer, I want to see past attendance statistics so I can evaluate engagement

Communication

- 77. As a student, I want threaded discussions for session pages, so that conversations are organized. (if time allows)
- 78. As a student, I want to reply to specific messages within a session, so that side conversations stay contained within the main discussion thread.
- 79. As a student, I want to receive notifications when someone replies to a thread I participated in, so that I can stay engaged without monitoring the entire chat.
- 80. As a student, I want to mute specific threads within a session, so that I can focus on conversations that matter to me.
- 81. As a student, I want to see a timestamp on messages in session discussions, so that conversations have context.
- 82. As a student, I want to search within a session's discussion threads, so that I can find specific information.
- 83. As a student, I want to react to messages with simple reactions, so that I can acknowledge messages without replying.
- 84. As a student, I want to delete messages I accidentally sent so others see a deletion notice, and I want the option to send auto-disappearing messages that vanish after a set time so sensitive content doesn't linger.
- ~~85. As a user, I want to send direct messages to another user and view my conversation history so that I can coordinate study plans privately.~~
- 86. As a user, I want to view my own and other users' public profiles so that I can learn more about potential study partners before connecting with them.

Non-Functional Requirements

Architecture and Performance

BoilerSpace will use a client server architecture based on the MERN stack (MongoDB, Express, Node.js) to support maintainability and future growth. The frontend will be a responsive web application built with React and Tailwind CSS, while the backend will provide a REST API built with Node.js and Express to manage user accounts, room availability, study sessions, file uploads, and AI-assisted queries. This separation allows system components to be developed and updated independently. The system will be designed to provide responsive performance under typical usage. Common actions such as viewing room availability, browsing study sessions, and loading the campus map should be completed within 1 second. More resource intensive actions including operations with the AI Knowledge Chatbot should be completed within

3 seconds. The application is expected to support approximately 300 concurrent users without noticeable performance degradation. Authentication and authorization will be implemented using Passport.js within our Express backend. User passwords will be salted and hashed using bcrypt (with a work factor of 10) before storage in MongoDB. To secure the REST API, we utilize the passport-jwt strategy, requiring a valid, signed JSON Web Token or JWT in the authorization header for all protected endpoints. This stateless authentication method prevents Cross-Site Request Forgery and ensures data scraping is blocked.

Crowdsourced Data Integrity (Security)

BoilerSpace relies on crowdsourced check-ins to provide live room availability and study environment indicators. Room data should be based on combined input from multiple users, while still allowing recent individual check-ins to influence results when limited data is available. The system should account for users leaving rooms by gradually reducing the impact of older check-ins over time, ensuring that availability indicators stay aligned with real campus conditions.

Usability

The BoilerSpace interface emphasizes clear organization to help users efficiently discover study spaces, sessions, and campus events. The interactive campus map enables users to explore locations, view ability, and access relevant information with minimal effort. Study sessions, courses, and club events are structured with filtering and search options to make content easy to find and sort. The AI chatbot is designed to be easily accessible within the application, allowing users to ask questions and review course related content without disrupting their workflow. Users should also be easily able to view past notes and materials shared by other students or generated through the AI assistant. Profile features such as class selection and visibility controls are streamlined so users can manage how they are represented to others. The application will provide a consistent and usable experience across devices, screen sizes, and modern web browsers.

Hosting/Deployment

To maintain continuous delivery and scalability, the application will use a split deployment strategy. The frontend will be hosted on Vercel, which provides automatic deployments from our GitHub repo and a global CDN for fast load times. The backend will be deployed on Render. This way we can implement our crowdsourcing and messaging features, which require a long-running server process.

Current Sprint Detail

User Story 17: Room Capacity Notifications

Story: As a student, I would like to receive notifications for when a room is under a certain capacity so I can travel to the location knowing I will successfully find a seat.

Tasks (Jed)

1. Design capacity threshold and notification preference schema — 2.0 hrs

Extend the User schema with a notification preferences sub-document that stores per-room or per-building capacity thresholds chosen by the user. Add fields for threshold value, target room/building reference, and enabled status.

2. Implement capacity monitoring and notification trigger logic — 3.0 hrs

Develop backend logic that evaluates live occupancy against user-set thresholds whenever a check-in or check-out event occurs. When a room drops below the threshold, queue a notification for each subscribed user.

3. Build notification delivery endpoint and frontend alert UI — 2.0 hrs

Create a GET /notifications endpoint that returns pending notifications for the authenticated user. Build a React notification bell component that displays unread alerts with room name, current occupancy, and timestamp.

4. Testing and validation coverage — 1.0 hr Write basic tests to verify notification preferences are saved correctly and that the notification endpoint returns alerts only for the authenticated user.

Estimated total: 8.0 hrs

Acceptance Criteria

- Given I am on a room's detail page, when I set a capacity threshold (e.g., "notify me when under 50%") and save, then I see a confirmation that my alert preference has been saved.
- Given I have set a capacity alert for a room, when that room's live occupancy drops below my threshold, then I see a notification appear in my notification bell with the room name and current occupancy.
- Given I receive a capacity notification, when I click on it, then I am taken to that room's detail page showing the current live occupancy data.

User Story 18: Location Access Explanation

Story: As a student, I want to see a clear explanation when location access is requested, so that I understand why it is needed before granting permission.

Tasks (Jed)

1. Design and build the location permission prompt UI — 2.0 hrs

Create a custom React modal that appears before the browser's native location prompt. The modal explains what location data is used for (sorting rooms by distance, showing nearby buildings), assures the user their location is not stored, and provides "Allow" and "Not Now" buttons.

2. Implement location permission state management — 2.0 hrs

Build frontend logic that tracks whether the user has granted, denied, or not yet responded to the location request. If denied, gracefully degrade location-dependent features (e.g., hide “sort by distance”) and show a settings hint explaining how to re-enable.

3. Integrate location prompt trigger with map and room list features — 1.5 hrs

Wire the custom prompt to fire only when the user first accesses a feature that needs location (e.g., opening the campus map or clicking “Sort by distance”). Ensure the prompt does not re-appear on every page load if the user has already responded.

4. Testing and validation coverage — 1.0 hr Write basic tests to verify that location-dependent features (sort by distance, nearby buildings) are hidden when permission is denied and visible when permission is granted.

Estimated total: 6.5 hrs

Acceptance Criteria

- Given I open the campus map, when the app needs my location, then I see a custom explanation modal describing why location access is needed before any browser prompt appears.
- Given I click “Allow” on the explanation modal, when the browser’s native location prompt appears and I grant access, then location-dependent features (sort by distance, nearby buildings) become available.
- Given I click “Not Now” on the explanation modal, when I continue using the app, then location-dependent features are hidden or disabled and I see a brief hint explaining I can enable location later in my browser settings.
- Given I have already granted or denied location access, when I navigate to the map again, then the explanation modal does not re-appear.

User Story 41: Directions to Study Locations

Story: As a student I want directions to different study locations so that I can find the fastest route to a building I haven’t visited before.

Tasks (Jed)

1. Integrate MapBox Directions API with the campus map component — 4.0 hrs

Add the MapBox Directions plugin to the existing map component. Configure it to use walking mode by default (appropriate for campus navigation) and display the route overlay with estimated walking time on the map.

2. Build “Get Directions” button and route display UI — 3.5 hrs

Add a “Get Directions” button to building detail popups and room cards. When clicked, use the user’s current location as the start point and the building’s coordinates as the destination. Display step-by-step walking directions in a collapsible side panel alongside the route on the map.

3. Handle edge cases for location unavailability — 2.5 hrs

If the user has not granted location access, show a prompt asking them to enable it (reusing the User Story 18 modal). If location cannot be determined, allow the user to manually enter or select a starting building from the directory as an alternative origin point.

4. Testing and validation coverage — 1.0 hr Write basic tests to verify the directions API returns a valid route and that the UI falls back to manual building selection when location is unavailable.

Estimated total: 11.0 hrs

Acceptance Criteria

- Given I am viewing a building on the map, when I tap “Get Directions,” then I see a walking route drawn on the map from my current location to that building along with the estimated walking time.
- Given a route is displayed, when I look at the side panel, then I see step-by-step walking directions I can follow to reach the building.
- Given I have not granted location access, when I tap “Get Directions,” then I am prompted to enable location or manually select a starting building from the directory.
- Given I select a different building as my starting point, when the route loads, then it shows the walking path between the two buildings with accurate distance and time.

User Story 34: Enhanced Dashboard and Room Pages

Story: As a student, I want my dashboard and room pages to display recent buildings I’ve visited, key room amenities (such as whiteboards and outlets), and the last updated time of room status so that I can quickly choose a study space that fits my needs and trust the availability information.

Tasks (Jed)

1. Add amenity fields to Room schema and seed amenity data — 2.0 hrs

Extend the Room data model with an amenities array (e.g., whiteboards, outlets, projector, natural lighting). Update the seeding script to populate amenity data for each room.

2. Track recent building visits and build the “Recently Visited” dashboard section — 3.0 hrs

Record a visit entry each time a user checks in or views a building detail page. Build a “Recently Visited” card row on the dashboard that shows the user’s last few buildings with quick links back to their detail pages.

3. Display amenity icons and last-updated timestamp on room pages — 2.5 hrs

Render amenity icons (whiteboard, outlet, etc.) on each room card and detail page. Add a “Last updated” timestamp that shows when the most recent check-in or status change occurred for that room, formatted as relative time (e.g., “5 min ago”).

4. Testing and validation coverage — 1.0 hr Write basic tests to verify amenity data displays on room cards and that the recently visited section populates after a user checks into a building.

Estimated total: 8.5 hrs

Acceptance Criteria

- Given I have previously checked into buildings, when I open my dashboard, then I see a “Recently Visited” section showing the last buildings I visited with quick links to their pages.

- Given I am viewing a room's detail page, when the page loads, then I see icons representing the room's amenities (whiteboards, outlets, etc.) so I know what equipment is available before I go.
- Given a room has recent check-in activity, when I view its page, then I see a "Last updated" timestamp (e.g., "5 min ago") so I can judge how current the occupancy information is.
- Given I click on a building in my "Recently Visited" section, when the page loads, then I see that building's rooms with their amenities and live occupancy data.

User Story 84: Delete Messages

Story: As a student, I want to delete messages I accidentally sent so others see a deletion notice, and I want the option to send auto-disappearing messages that vanish after a set time so sensitive content doesn't linger.

Tasks (Aagam)

1. **Extend Message schema with deletion and expiry fields — 2.0 hrs** Add soft-delete fields and a disappearing message flag with an expiry timestamp to the Message model. Set up a TTL index so expired messages are automatically removed by MongoDB.
2. **Implement message deletion backend logic — 2.5 hrs** Build an endpoint so only the message author can delete their message. Instead of removing it, replace the content with a "This message was deleted" placeholder to preserve thread structure.
3. **Implement auto-disappearing message logic — 3.0 hrs** When a message is sent with a disappearing flag, store the expiry time. Run a background job that cleans up expired messages and notifies all participants in the conversation when one disappears.
4. **Build frontend deletion and disappearing timer UI — 2.5 hrs** Add a delete option to messages the current user sent. Show a placeholder where deleted messages were. Add a timer toggle in the message composer so users can set a disappearing duration before sending.
5. **Testing and validation coverage — 2.0 hrs** Test that only authors can delete, non-authors are blocked, placeholder renders correctly, and disappearing messages are removed after expiry.

Estimated total: 12.0 hrs

Acceptance Criteria:

- Given I sent a message, when I click delete, then it is replaced with "This message was deleted" for all participants.
- Given I did not send a message, I can not delete it. (I can not delete others messages)
- Given I send a disappearing message, when the timer runs out, then the message is gone for everyone.
- Given I open the message composer, when I toggle the disappearing option, then I can choose how long the message lasts before it is deleted.

User Story 13: Account Deletion

Story: As a user, I want to be able to delete my account so that I can remove my personal data if I leave the university or stop using the app, and we will also have 2FA through email OTP.

Tasks (Aagam)

1. **Implement account deletion with cascading cleanup — 3.0 hrs** Build a delete account endpoint that removes all of the user's data including messages, notes, bookmarks, check-ins, and friendships. Invalidate their active sessions so old tokens stop working immediately.
2. **Build 2FA via email OTP — 3.0 hrs** After a user logs in with correct credentials, generate a 6-digit OTP and send it to their Purdue email. Block access to the dashboard until the OTP is successfully verified.
3. **Build frontend account deletion flow — 2.0 hrs** Add a "Delete Account" button in account settings. Show a confirmation modal where the user must type their email before deletion is allowed.
4. **Build frontend 2FA verification screen — 1.0 hr** Add an OTP entry screen that appears after successful login. Redirect to the dashboard once the code is verified.
5. **Testing and validation coverage — 2.0 hrs** Test cascading deletes, session invalidation after deletion, OTP expiry, and the confirmation modal flow.

Estimated total: 11.0 hrs

Acceptance Criteria:

- Given my account is deleted, when I try to log in, then I see an error saying there is no email or password associated with the account.
- Given my account is deleted, I can use the email and password that was deleted to sign up
- Given I want to delete my account, it will send an OTP to my email to confirm I want to delete it
- If I enter the wrong OTP I will not be able to delete my account, and if I enter the correct one it will delete it
- If I delete my account if someone tries to message me they will get an error

User Story 85: Direct Messaging

Story: As a user, I want to send direct messages to another user and view my conversation history so that I can coordinate study plans privately.

Tasks (Aagam)

1. **Design Conversation and Message schemas — 2.0 hrs** Create data models for conversations and messages. A conversation tracks participants and the last message. A message stores the sender, content, and which conversation it belongs to.
2. **Build conversation and message backend logic — 3.0 hrs** Build endpoints to start or retrieve a conversation between two users, list all conversations for the current user sorted by most recent, send a message, and fetch paginated message history.
3. **Add real-time messaging via WebSocket — 2.5 hrs** Set up Socket.io so messages are delivered instantly to online participants. Make sure blocked users cannot receive messages or socket events from users who blocked them.
4. **Build frontend DM inbox and chat UI — 2.5 hrs** Create an inbox showing all conversations with the other user's name, avatar, last message preview, and unread count. Build a chat window with scrollable history and a message input.
5. **Testing and validation coverage — 2.0 hrs** Test conversation creation, message pagination, blocked user enforcement, and real-time delivery.

Estimated total: 12.0 hrs

Acceptance Criteria:

- Given I open my inbox, when it loads, then I see all my conversations sorted by most recent.
- Given I send a message, when it is sent, then it appears in the chat.
- Given I reopen a conversation, when it loads, then I can see previous messages.
- Given I open a conversation, when it loads, then I can see the other user's name and profile picture.

User Story 42: Rooms Sorted by Distance

Story: As a student, I want to see a list of rooms sorted by distance from my current location so that I can find the nearest available spot.

Tasks (Bidit)

1. Implement Geolocation API – 2.0 hours:

Implement browser Geolocation API to capture user's coordinates with latitude and longitude and implement proper permission handling

2. Create Distance Location Logic – 2.5 hours:

Use Haversine's formula for shortest path over Earth's surface to calculate the distance between users and study spaces.

3. Build Sorted Results Endpoints – 2.0 hours:

Create backend route that accepts coordinates as parameters and returns a list of buildings sorted by distance.

4. Update UI to Display Distance – 2.0 hours:

Modify the building list component to order itself by distance and also display distance either in "miles away" or "minutes away."

5. Testing and Validation Coverage – 1.5 hours:

Test distance calculation, building sorting, and geolocation API success.

Estimated total: 10.0 hours

Acceptance Criteria:

- Given I grant location permissions, when the building list loads, then I see the closest buildings at the top of the list.
- Given I am looking at a room card, when the list is populated, then I can see exactly how far that building is from my current location.
- Given my location changes, when I refresh the list, then the distances and sorting update accordingly.

User Story 54: Filter Rooms by Amenities

Story: As a student, I want to filter rooms based off amenities (whiteboards, outlets) so that I can find a workspace that fits my equipment needs.

Tasks (Bidit)

1. Update Room schema – 2.5 hours:

Modify Room schema to include amenities individually instead of storing all in a list, making it easier to filter.

2. Build Filter Backend Endpoint – 3.0 hours:

Develop a GET endpoint that accepts query parameters (e.g. checking if filtered amenity is a whiteboard) and returns a list of all rooms with those amenities.

3. Create Filter UI – 3.0 hours:

Design a multi-select checkbox sidebar next to the current rooms sidebar that allows users to see which amenities to filter for.

4. Testing and Validation Coverage – 1.5 hours:

Test that multiple filters work at the same time and ensure proper error messages are shown.

Estimated total: 10.0 hours

Acceptance Criteria:

- Given I am looking for a room, when I select "whiteboard" in the filter menu, then only rooms equipped with whiteboards are displayed.
- Given I select multiple filters (e.g. "outlets" and "whiteboard"), when the list filters, then only rooms containing both amenities are shown.
- Given I clear my filters, when the page updates, then I see the full unfiltered list of available rooms.

User Story 57: AI Study Guide Generation

Story: As a student, I want the AI assistant to generate concise study guides from my notes and highlight key topics, so I can review efficiently and know what to focus on.

Tasks (Bidit)

1. Setup GCS for Note Retrieval – 2.0 hours:

Set up Google Cloud Storage so users can download notes uploaded by other users, and also for the Gemini API to access the contexts of the notes for study guide generation.

2. Implement AI Generation Backend – 4.0 hours

Create API endpoint that retrieves user notes for a class, sends them to Gemini, and returns a comprehensive study guide based on a hard-coded prompt in the backend.

3. Build Study Guide UI – 2.5 hours:

Add a "Generate Study Guide" button to the notes page and a separate modal for the study guide once it gets generated, and give users the option to regenerate.

4. Testing and Validation Coverage – 1.5 hours:

Test for successful note parsing, study guide creation, and proper formatting.

Estimated total: 10.0 hours

Acceptance Criteria:

- Given I have a set of uploaded notes, when I click "Generate Study Guide," then I receive a comprehensive overview of the material.
- Given a generated guide, when I view the "Key Topics" section, then the most important concepts from my notes are highlighted or bolded.
- Given a new guide is generated, when I refresh the page, then I can still access the previously generated guide in my history.

User Story 58: AI Practice Questions

Story: As a student, I want the AI to suggest practice questions without giving full solutions, so that I can test my understanding.

Tasks (Nathan)

- **Design AI prompt structure and constraints — 3.0 hrs**
Define how course notes and user input are formatted into prompts that generate practice questions while preventing the AI from revealing full solutions.
- **Implement backend endpoint for AI question generation — 3.0 hrs**
Develop a backend API route that sends prompts to the AI model and returns generated practice questions based on the selected course notes.
- **Build frontend interface for requesting practice questions — 2.5 hrs**
Create UI components allowing students to request practice questions and view generated results in a clear format.
- **Testing and validation coverage — 1.5 hrs**
Add tests to ensure prompts generate questions correctly and verify that responses do not include full solutions.

Estimated total: 10.0 hrs

Acceptance Criteria

- Given I am viewing a course page, when I request AI-generated practice questions, then I see a list of practice questions related to the course material.
- Given I receive an AI-generated practice question, when the question first appears, then I only see the question and not the answer.
- Given I want to check my work, when I click the “Reveal Answer” button, then I can see the explanation or solution to the practice question.
- Given I have revealed the answer, when I view the explanation, then it is clearly displayed and easy for me to understand.

User Story 55: AI Course Q&A

Story: As a student, I want to ask AI assistant questions about the aggregated study notes for my course so that I can find specific explanations without reading every document.

Tasks (Nathan)

- **Design note aggregation and AI prompt pipeline — 3.0 hrs**
Define how course notes are combined and formatted so the AI can answer questions using the relevant study material.
- **Implement backend endpoint for AI course Q&A — 3.0 hrs**
Develop an API route that sends the aggregated notes and student question to the AI model and returns a response.
- **Build frontend interface for asking AI questions — 2.5 hrs**
Create UI components allowing users to submit questions and view AI responses tied to their course notes.
- **Testing and validation coverage — 1.5 hrs**
Add tests to verify correct API responses and ensure the AI answers are generated using course-related content.

Estimated total: 10.0 hrs

Acceptance Criteria

- Given I am viewing my course notes, when I ask the AI assistant a question about the material, then I receive an explanation related to my course notes.

- Given I submit a question to the AI assistant, when the response appears, then I can clearly read the answer on the page.
- Given I ask a question and there are no relevant notes available, when the AI attempts to answer, then I see a message explaining that there is not enough information.
- Given I submit a question and the AI service fails, when the request cannot be processed, then I see a message explaining that my question could not be answered.

User Story 73: Club Member Approval and Removal

Story: As a club organizer, I want to approve or remove club members, so that organizations stay moderated.

Tasks (Nathan)

- **Design membership approval schema — 2.5 hrs**
Define database fields to track membership status including pending, approved, and removed states.
- **Implement member approval API endpoint — 3.0 hrs**
Create backend route allowing organizers to approve membership requests for their club.
- **Implement member removal functionality — 2.5 hrs**
Develop endpoint allowing organizers to remove members from a club if necessary.
- **Testing and authorization validation — 2.0 hrs**
Add tests to verify that only authorized organizers can approve or remove members.

Estimated total: 10.0 hrs

Acceptance Criteria

- Given I am a club organizer, when a student requests to join my club, then I can see their request and choose to approve it.
- Given I approve a membership request, when the action is completed, then I see the student listed as a member of the club.
- Given I am a club organizer, when I remove a member from my club, then that student no longer appears in the club member list.
- Given I am not the club organizer, when I try to approve or remove members, then I see a message telling me that I do not have permission.

User Story 9: Friend Users by Class

Story: As a user, I want to be able to friend others based on what classes they are taking so that I can build a network of study partners for my specific coursework.

Tasks (Rohan)

- **Design User-Course Query and Filtering Logic — 2.5 hrs**
Develop optimized MongoDB queries to filter the `User` collection based on shared `courseID` arrays. This includes logic to handle "mutual" class discovery while respecting user privacy flags.
- **Implement 'Classmate Discovery' API Endpoint — 3.5 hrs**

Create a `GET /users/discovery/courses` endpoint in the `SocialController` that returns a list of students enrolled in the same sections, including their major and year as defined in the Class Level Design.

- **Integrate with Friendship Request System — 3.0 hrs**

Connect the discovery results to the `Friendship` schema. Implement logic to check for existing pending requests to prevent duplicate "Friend" invitations from the course list view.

- **Testing, Privacy, and Data Validation — 2.0 hrs**

Write unit tests to ensure users who opted out of "Class Discovery" do not appear in results and verify that the endpoint correctly paginates large class rosters.

Estimated total: 11.0 hrs

Acceptance Criteria

- Given I have added "CS 30700" to my current classes, when I view the class directory, then I see a list of other students enrolled in the same course.
- Given I see a classmate in the course list, when I click "Add Friend," then a friend request is sent to them and a notification is triggered.
- Given I am looking at a classmate's profile from the discovery list, when I click their name, then I am taken to their profile view to see their shared interests before friending them.
- Given a user has their privacy settings configured to hide their courses, when I search by class, then their profile does not appear in the results.
- Given I receive a friend request from a classmate, when I accept it, then we become mutual friends and gain full visibility of each other's profiles.

User Story 12: Profile Picture Upload

Story: As a user, I want the option to upload a profile picture so others can recognize me more easily.

Tasks (Rohan)

- **Update User Schema and Image Sanitization — 2.0 hrs**

Modify the `User` model to include `profilePictureUrl`. Implement backend logic to sanitize filenames and generate unique identifiers to prevent overwriting files in storage.

- **Implement Google Cloud Storage Integration — 4.0 hrs**

Configure the `GCS` client library to handle stream uploads. Develop a utility service to upload buffers directly to the bucket and return the public URL for the database.

- **Develop Profile Image Update API — 3.0 hrs**

Update the `PUT /users/:id/profile-picture` endpoint. This includes integrating `multer` for multipart/form-data handling and logic to delete the "old" profile picture from the cloud when a new one is uploaded.

- **Image Validation and Integration Testing — 2.0 hrs**

Implement strict validation for file dimensions and MIME types (JPEG/PNG). Write integration tests to ensure the URL stored in MongoDB is reachable and matches the cloud object.

Estimated total: 11.0 hrs

Acceptance Criteria

- Given I am on the Edit Profile page, when I select a valid image file and click save, then my profile picture is updated across the application.
- Given I attempt to upload a file that is not an image (e.g., a PDF), when I submit the form, then I receive an error message stating the file type is unsupported.
- Given I have updated my profile picture, when I appear in a search result for a course, then my new profile picture is displayed next to my name.
- Given I have successfully uploaded a profile picture, when I post a message in a study group chat, then my new image appears next to my name.
- Given I want to remove my profile picture, when I edit my profile and remove the image, then my profile reverts to the default avatar.

User Story 47: Comment on Session Notes

Story: As a student, I want to comment on a session's notes so that I can ask clarifying questions or provide corrections to shared content.

Tasks (Rohan)

- **Design Note-Discussion Database Schema — 2.5 hrs**
Create a `NoteComment` schema that references both the `Note` ID and the `User` ID. Ensure comments are indexed by `Note` ID for high-performance retrieval during session viewing.
- **Implement Comment Creation API — 3.5 hrs**
Develop the `POST /notes/:id/comments` endpoint. Implement authorization checks to ensure the user is part of the `StudySession` or `Course` before they are allowed to post.
- **Develop Threaded Retrieval Logic — 3.0 hrs**
Create the `GET /notes/:id/comments` endpoint. Implement sorting logic to return comments in chronological order and include user metadata (name/avatar) for the frontend display.
- **Testing and Real-time Validation — 1.5 hrs**
Write tests to verify that deleting a note also cleans up its associated comments and ensure that empty or malicious scripts in comments are sanitized.

Estimated total: 10.5 hrs

Acceptance Criteria

- Given I am viewing a shared PDF note in an active study session, when I type a comment and submit it, then my comment appears in the sidebar with my name and a timestamp.
- Given a note has existing comments, when I open the note viewer, then I can see a chronological history of all previous questions and replies.
- Given I want to reply to a specific classmate's question on a note, when I send a message, then my comment is added to the shared discussion thread for that session.
- Given I am not a participant in a private study session, when I try to access its notes and comments, then the system denies me access and displays an error.

User Story 10: Update/Add preferred study times

Story: As a user, I want to update my availability or preferred study times, so that others know when I'm typically open to studying.

Tasks (Sheehan)

- **Design Availability Schema & Validation - 2.5 hrs**
Modify `backend/models/User.js` to add a new `availability` field (array of `{ day, startTime, endTime }` objects) and implement validation logic in `backend/utils/timeValidation.js` to make sure time ranges formatted correctly.
- **Implement Availability API Endpoints - 3.5 hrs**
Create `GET /api/users/me/availability` and `PUT /api/users/me/availability` routes in `backend/routes/users.js` with controller functions in `backend/controllers/userController.js` that authenticate the user with `middleware/auth.js` and update the `availability` field in the database.
- **Build Frontend Availability Editor Component - 3.0 hrs**
Add React component `frontend/src/components/profile/AvailabilityEditor.jsx` that displays editable time slots for each day and sends updated availability to the backend using a `PUT` request to `/api/users/me/availability` from `frontend/src/services/userApi.js`.
- **Testing, Edge Cases & Documentation - 2.0 hr**
Add backend tests in `backend/tests/userAvailability.test.js` and frontend component tests in `frontend/src/tests/AvailabilityEditor.test.jsx` to verify valid updates, rejection of invalid times, and correct API communication.

Estimated total: 11 hrs

Acceptance Criteria

- Given logged-in user, when they open Profile - Availability, then the editor displays the currently saved availability (days, ranges, timezone) fetched from `GET /users/me/availability`.

- Given logged-in user, when they enter one or more valid day/time ranges and Save, then `PUT /users/me/availability` returns `200 OK`, the database is updated, and UI shows the new availability.
- Given user has saved availability, when they refresh or reopen their profile, then displayed availability matches what was saved (data persisted).
- Given the user inputs invalid ranges (e.g., `start >= end`, overlapping ranges, bad format), when they try to save, then the API returns `400` and the UI displays specific validation errors; the database is not updated.
- Given a request without a valid token or a user trying to update another user's availability, when `PUT /users/:id/availability` is called, then the server returns `401` or `403` and no update occurs.

User Story 26: Timestamp for Last Update for Room Status

Story: As a student, I want to see a timestamp of last update for a room status so I can judge how reliable the information is before I start walking

Tasks (Sheehan)

- **Schema + Data Flow - (2.5 hrs)**

Add a `lastStatusUpdate: Date` field to `backend/models/Room.js` (or `last_status_update` in the Room document); update `backend/models/Checkin.js` so check-ins record `createdAt` and ensure check-in creation code sets `room.lastStatusUpdate = new Date()`.

- **Backend: Status Endpoint & Aggregation Logic - (3.5 hrs)**

Implement `GET /api/rooms/:roomId/status` in `backend/routes/rooms.js` / `backend/controllers/roomController.js` to return `{ occupancy, lastStatusUpdate }`, where `lastStatusUpdate` is taken from `room.lastStatusUpdate`.

- **Frontend: UI Display & Formatting - (2.5 hrs)**

Update `frontend/src/components/rooms/RoomCard.jsx` and `frontend/src/components/rooms/RoomDetail.jsx` to show a small "Last updated: X ago" line using a utility `frontend/src/utils/timeUtils.js` (function `formatRelative(date)`), with a tooltip showing the absolute timestamp.

- **Tests & Documentation - (2 hrs)**

Add backend unit/integration tests in `backend/tests/roomStatus.test.js` verifying endpoint returns `lastStatusUpdate` and updates after check-in; add frontend tests in `frontend/src/tests/RoomCard.test.jsx` for rendering; update `docs/API.md` with `GET /api/rooms/:roomId/status` example response.

Estimated total: 10.5 hrs

Acceptance Criteria

- Given the user views a room card or room detail page, when the room status is fetched from the server, then the UI displays a relative timestamp which corresponds to the server `lastStatusUpdate` value.

- Given the relative timestamp is visible, when the user hovers or taps the timestamp, then a tooltip shows the absolute timestamp including date, time, and timezone.
- Given an authenticated user performs a room check-in (POST `/api/rooms/:roomId/checkin`), when the check-in completes, then `room.lastStatusUpdate` is set to a timestamp within 5 seconds of the check-in and `GET /api/rooms/:roomId/status` returns the new timestamp.
- Given a room has no check-ins and no `lastStatusUpdate`, when the UI displays the room, then the timestamp area shows a placeholder such as `Last updated: N/A`.

User Story 36: Snacks Icon with Crowdsourcing

Story: As a student, I want to see a small icon next to a building name that indicates if it has a cafe or vending machines nearby so I can choose a study spot based on snack availability and have a way to crowdsource that data.

Tasks (Sheehan)

- **Schema & Aggregation Logic - 3.0 hrs**

Update `backend/models/Building.js` to add a `snackReports` subdocument array and aggregated fields (`cafeScore`, `vendingScore`, `cafeCount`, `vendingCount`, `lastSnackReportAt`) and implement `backend/utils/aggregateSnackReports.js` to update the aggregated fields atomically whenever a report is added or removed.

- **Backend API Endpoints & Auth - 3.0 hrs**

Add routes in `backend/routes/buildings.js` and controller functions in `backend/controllers/buildingController.js`: GET `/api/buildings/:id/snacks` - returns aggregated snack info and recent reports. POST `/api/buildings/:id/snacks` - accepts `{ type: "cafe"|"vending", value: true|false }` and requires valid JWT `middleware/auth.js`, saves a new report, then calls the aggregator to update scores. Prevent duplicate immediate reports from the same user.

- **Frontend: Icon + Reporter UI - 2.5 hrs**

Create `frontend/src/components/buildings/SnackIcon.jsx` to render the small icon (cafe/vending) next to building names and `frontend/src/components/buildings/SnackReporter.jsx` to let authenticated users submit reports. Hook to `frontend/src/services/buildingApi.js` to call GET/POST `/api/buildings/:id/snacks`. Show with aggregated status.

- **Tests, Edge Cases & Docs - 2 hrs**

Write backend tests in `backend/tests/buildingSnack.test.js` to verify reporting, aggregation, and dup prevention; write frontend tests in `frontend/src/tests/SnackIcon.test.jsx` to verify icon rendering and reporter behavior; add API example docs in `docs/API.md`.

Estimated total: 10.5 hrs

Acceptance Criteria

- Given a building has aggregated report data, when the building list or map loads, then the SnackIcon displays the correct visual state (present/absent/unknown) computed from `GET /api/buildings/:id/snacks`.
- Given an authenticated user, when they open the SnackReporter and submit `Present` or `Not present`, then `POST /api/buildings/:id/snacks` returns `200` and the aggregated `cafeScore/vendingScore` shown in the UI updates within ~2s.
- Given a user submits a report, when the server responds, then a subsequent `GET /api/buildings/:id/snacks` includes that report in `recentReports` and counts are incremented accordingly.
- Given a user tries to submit the same type report repeatedly within 1 hour, when they submit, then the server rejects or ignores duplicates and the counts do not increment (response `409` or message: "You recently reported this").

Summary of Assignments

- **Aagam:**

- User Story 13: Account Deletion & 2FA
- User Story 84: Delete Messages
- User Story 85: Direct Messaging
- **Bidit:**
 - User Story 42: Rooms Sorted by Distance
 - User Story 54: Filter Rooms by Amenities
 - User Story 57: AI Study Guide Generation
- **Jed:**
 - User Story 17: Room Capacity Notifications
 - User Story 18: Location Access Explanation
 - User Story 41: Directions to Study Locations
 - User Story 34: Enhanced Dashboard & Room Pages
- **Nathan:**
 - User Story 58: AI Practice Questions
 - User Story 55: AI Course Q&A
 - User Story 73: Club Member Approval & Removal
- **Rohan:**
 - User Story 9: Friend Users by Class
 - User Story 12: Profile Picture Upload
 - User Story 47: Comment on Session Notes
- **Sheehan:**
 - User Story 10: Update/Add preferred study times
 - User Story 26: Timestamp for Last Update for Room Status
 - User Story 36: Snacks Icon with Crowdsourcing